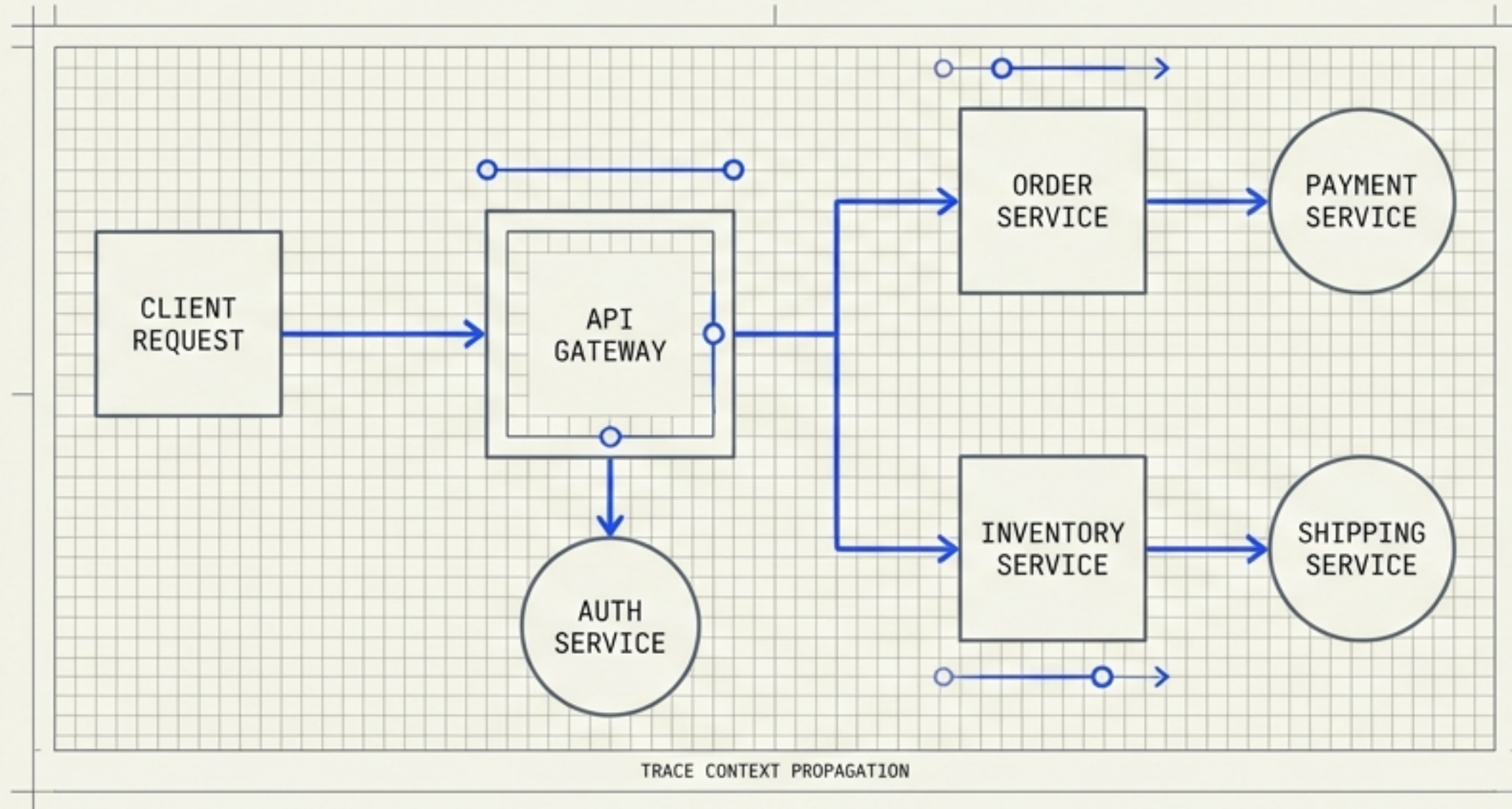


Iluminando la Caja Negra

Implementación Definitiva de Tracing Distribuido en Spring Boot 4

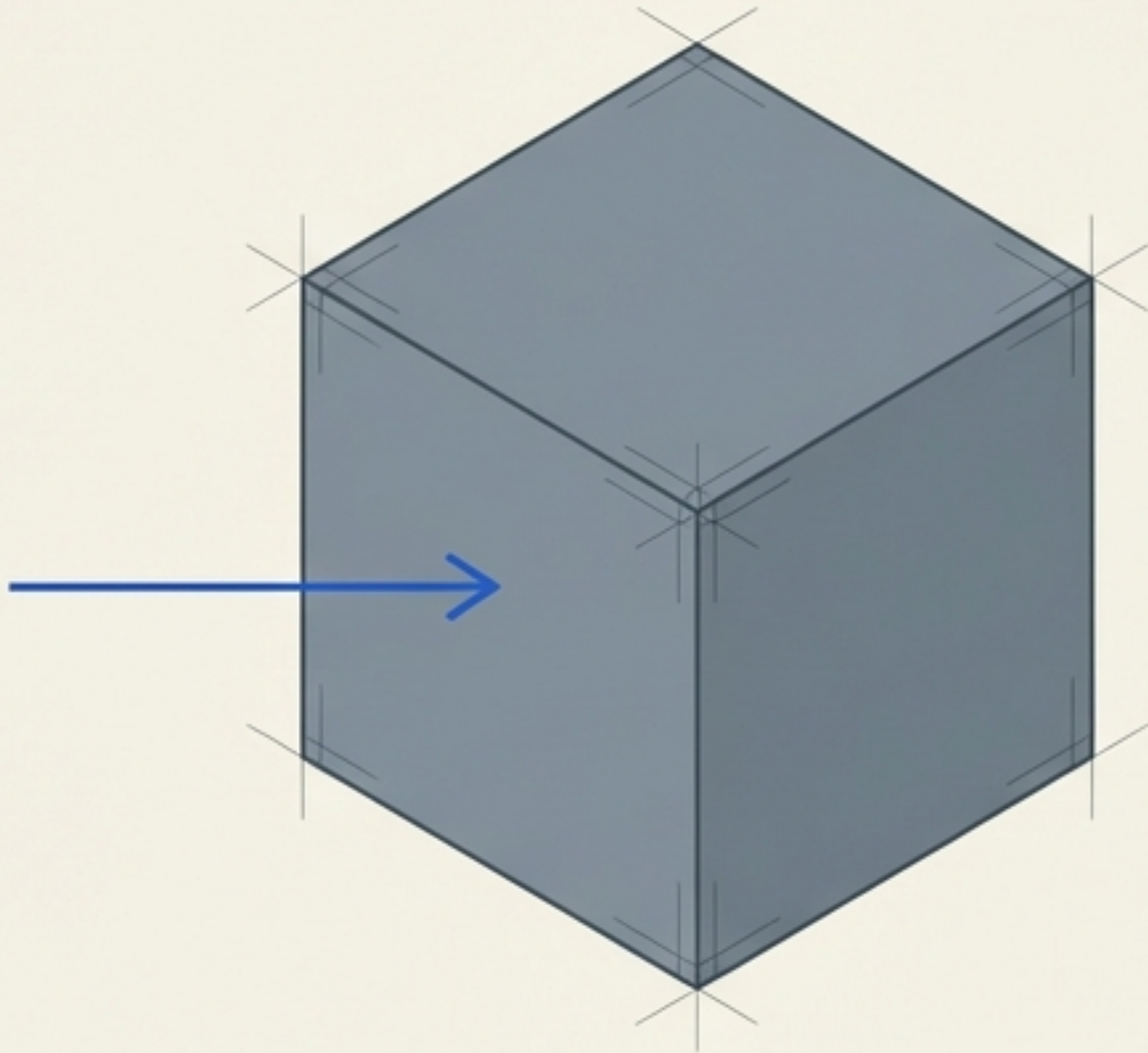
Una guía arquitectónica sobre OpenTelemetry, Micrometer y la propagación de contexto.



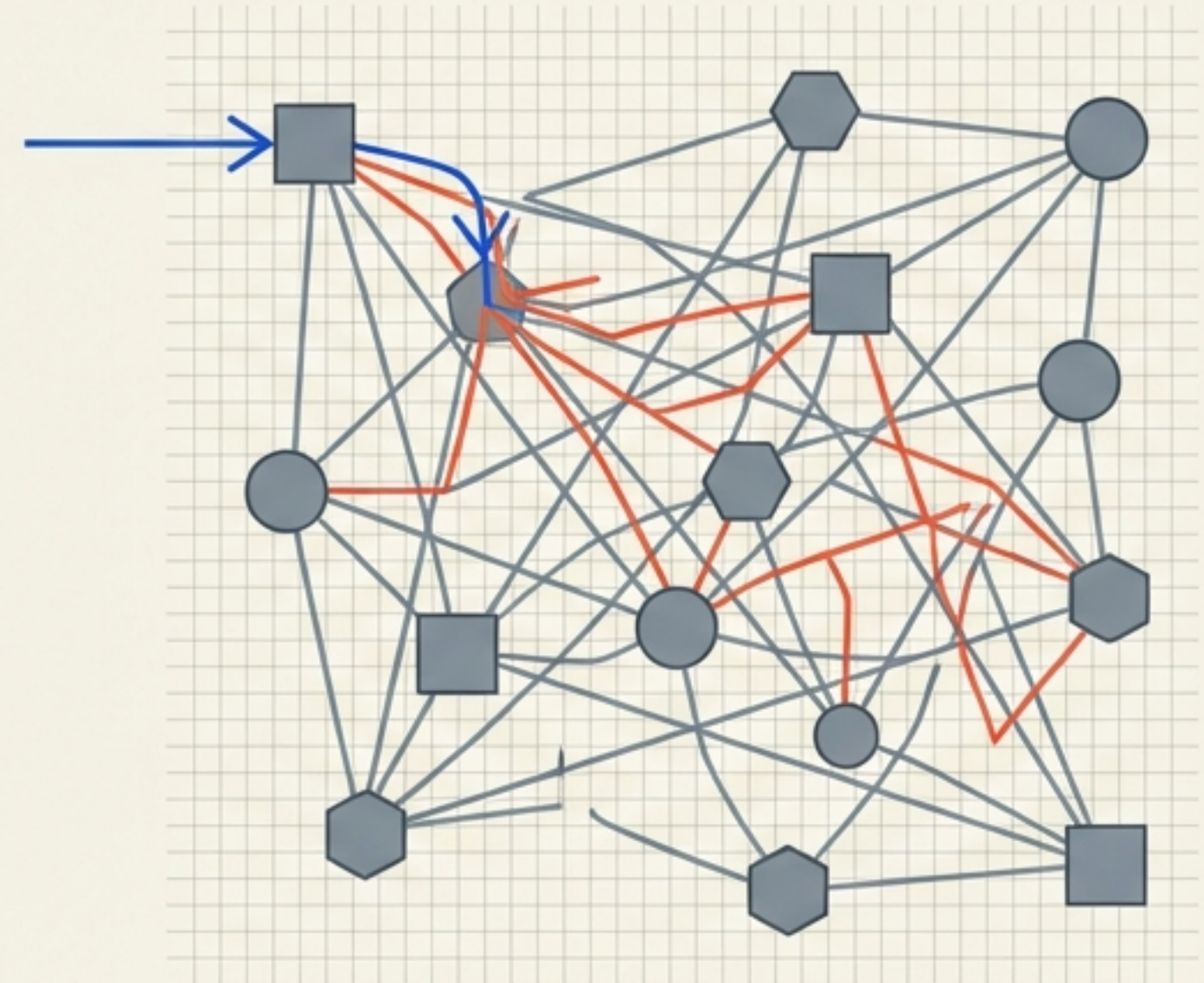
El Caos de los Microservicios

Sin observabilidad, una arquitectura distribuida es una caja negra. Cuando un request falla o se degrada, es imposible determinar qué nodo de la red es el culpable sin recorrer ciegamente los logs de docenas de servicios.

Antes (Monolito)



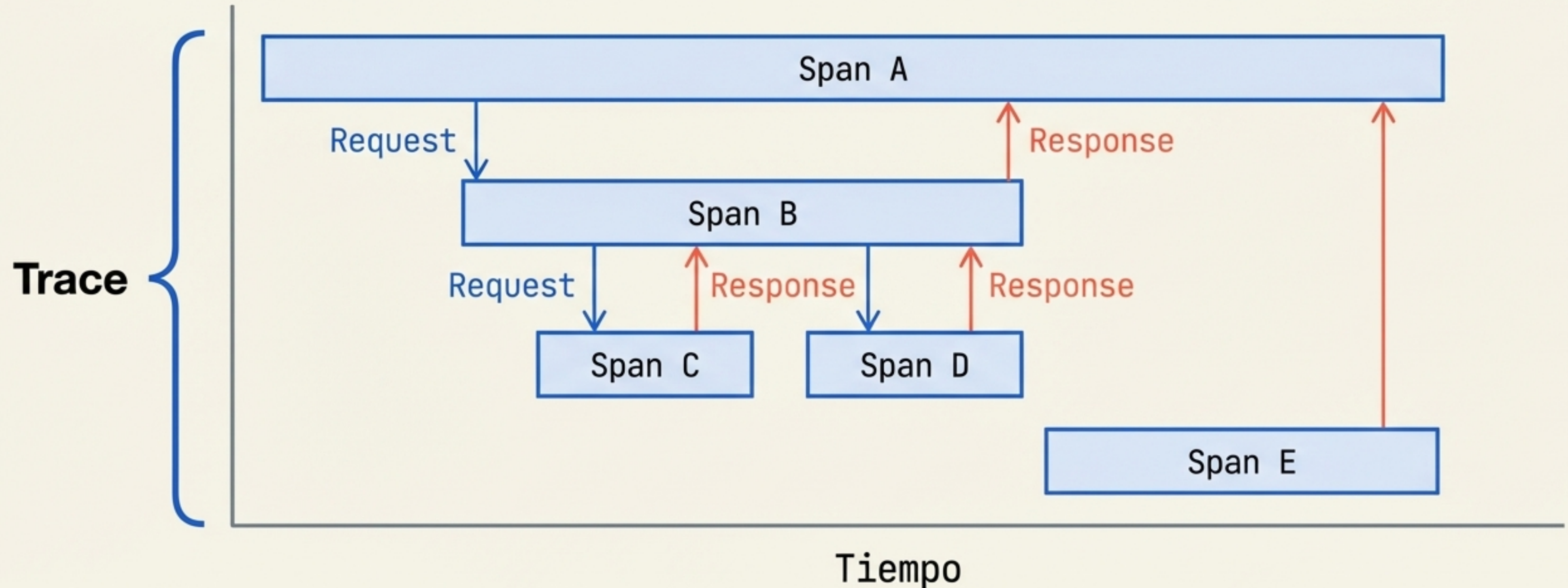
Después (Microservicios)



Anatomía de una Petición Distribuida

Trace: La ruta completa de ejecución de una solicitud a través de un conjunto de servicios.

Span: Una unidad lógica de trabajo (una consulta a BD, una llamada HTTP). Posee nombre, hora de inicio, duración y, crucialmente, una referencia a su Span padre.



La Frontera Arquitectónica: OpenTelemetry vs. Micrometer

Spring utiliza ambos. Entender sus responsabilidades es vital para una instrumentación correcta.

Micrometer	OpenTelemetry (OTel)
- Enfoque: Recolección de Métricas dimensionales (Counters, Timers).	- Enfoque: Tracing distribuido, propagación de contexto y Logs.
- RoI en Spring: La fachada nativa (el SLF4J de las métricas).	- RoI en Spring: El estándar agnóstico de exportación (OTLP).
- Overhead: Extremadamente bajo (<1% CPU).	- Overhead: Moderado (2-5% CPU).
- Estándar: Ecosistema Spring.	- Estándar: CNCF / W3C Trace Context.

El Puente: micrometer-tracing-bridge-otel traduce las observaciones nativas de Spring al formato universal de OTel.

Evolución y Migración: De Spring Boot 3 a 4

Spring Boot 4 introduce una modularización drástica para eliminar conflictos de dependencias y simplificar la exportación OTLP.

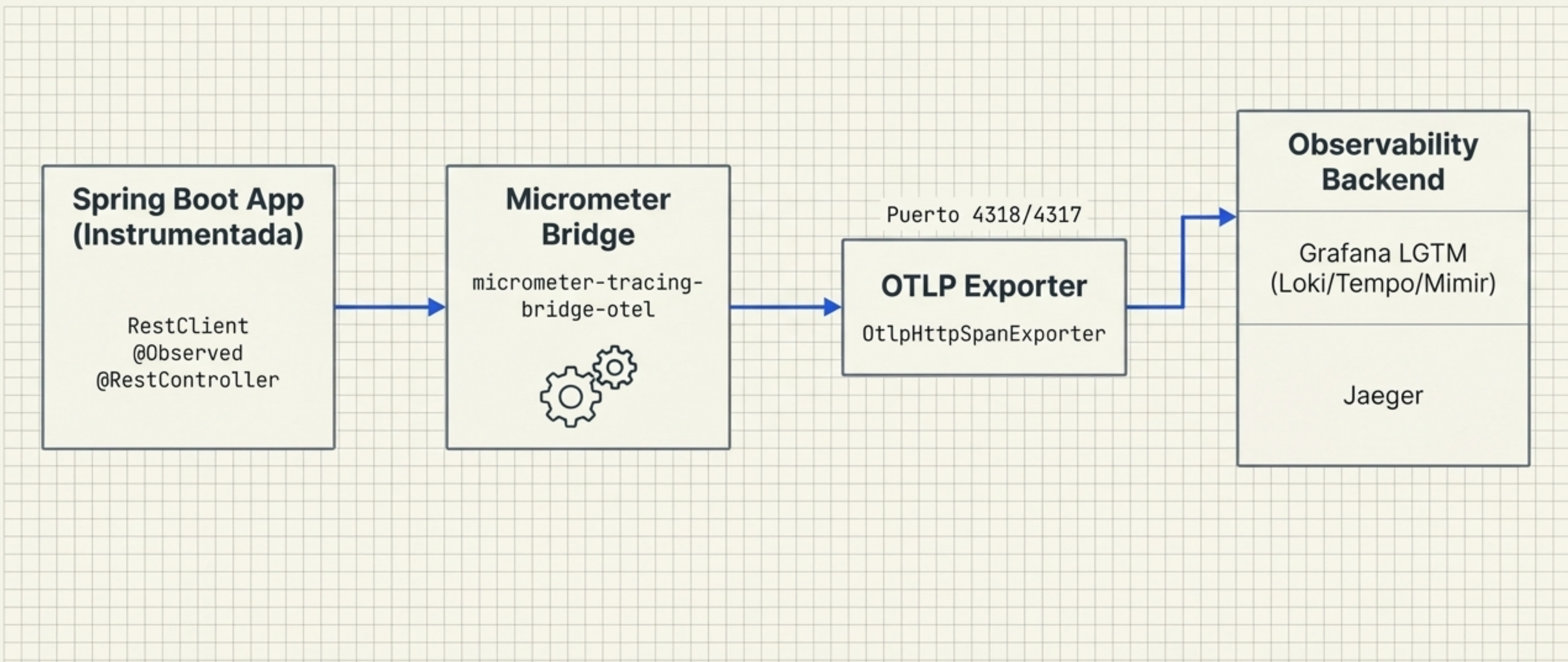
El Pasado (Spring Boot 2/3)

- ✗ Dependencias fragmentadas (spring-cloud-sleuth, agentes Java de terceros).
- ✗ Auto-configuración incompleta (riesgo de `ClassNotFoundException`).
- ✗ Propiedades legadas: `management.zipkin.tracing.*`.

El Estándar (Spring Boot 4)

- ✓ Starter unificado: `spring-boot-starter-opentelemetry`.
- ✓ Módulo dividido: `spring-boot-micrometer-tracing` se separó explícitamente en `-brave` y `-opentelemetry`.
- ✓ Propiedades normalizadas: `management.tracing.export.otlp.enabled`.
- ✓ Soporte AOT/GraalVM nativo.

El Pipeline de Datos OTLP en Spring Boot 4



Paso 1: El Arma Mágica (Dependencias)

El nuevo starter oficial inyecta la API de OpenTelemetry, el puente de Micrometer y los exportadores OTLP sin necesidad de agentes Java externos.

Auto-instrumentación: Cubre automáticamente RestClient, WebClient, llamadas JDBC y controladores HTTP.

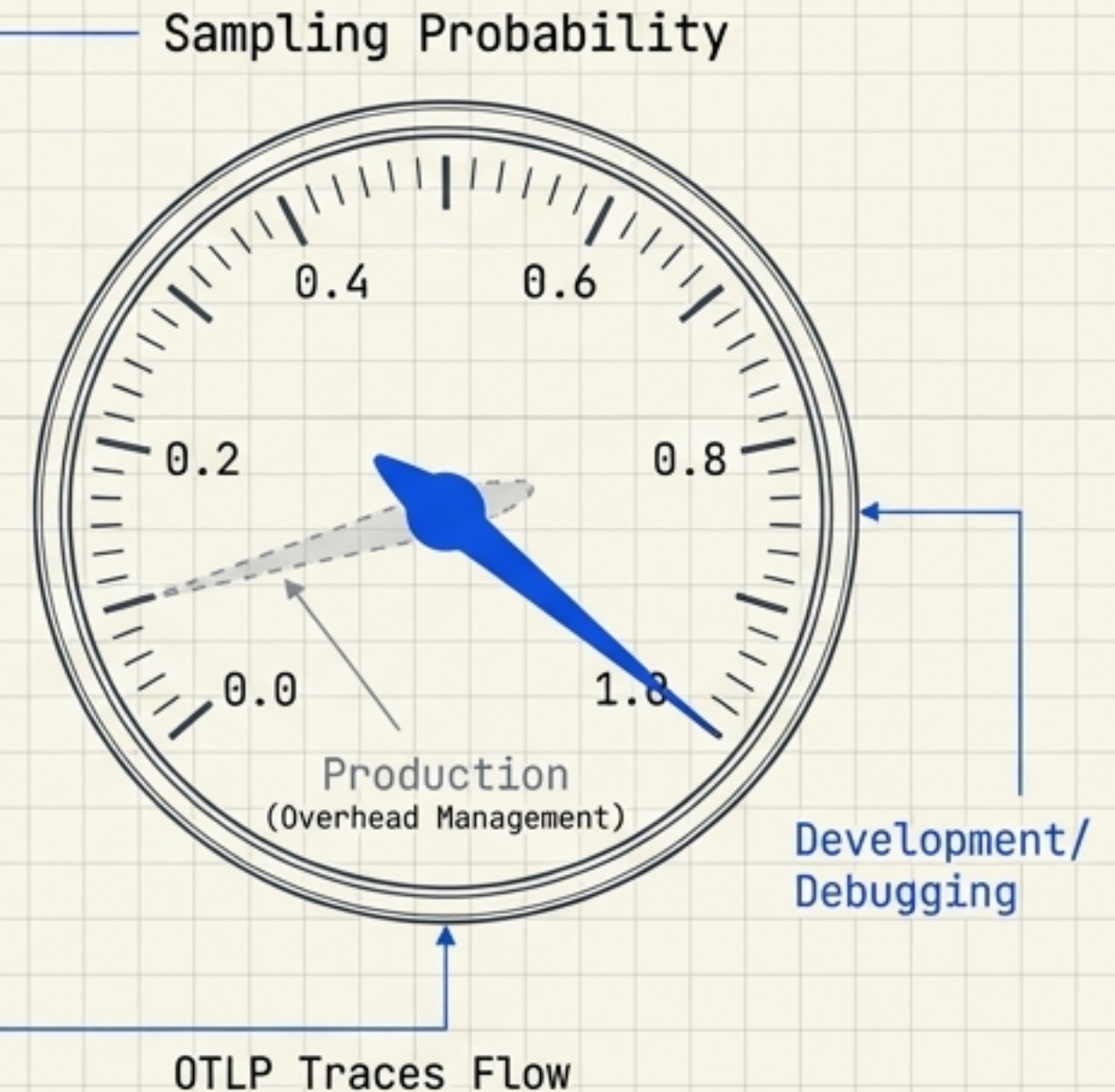
```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-opentelemetry</artifactId>  
</dependency>
```

Sin Agentes: Evita los conflictos de bytecode del opentelemetry-javaagent.jar.

Paso 2: Enrutamiento y Muestreo (Configuración)

Configuración nativa para definir qué porcentaje de tráfico analizar y hacia dónde enviar la telemetría.

```
spring:
  application:
    name: order-service
management:
  tracing:
    sampling:
      probability: 1.0 # 100% en DEV. Usar 0.1 en PROD.
opentelemetry:
  tracing:
    export:
      otlp:
        endpoint: http://localhost:4318/v1/traces
```



Paso 3: Unificando Logs y Traces

Spring Boot 4 ajusta automáticamente el patrón de logs, pero requiere un appender específico para exportarlos vía OTLP.

```
<appender name="OTEL" class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender" />
```

2024-12-23 INFO dev.danvega.ot.HomeController : Greeting user: World



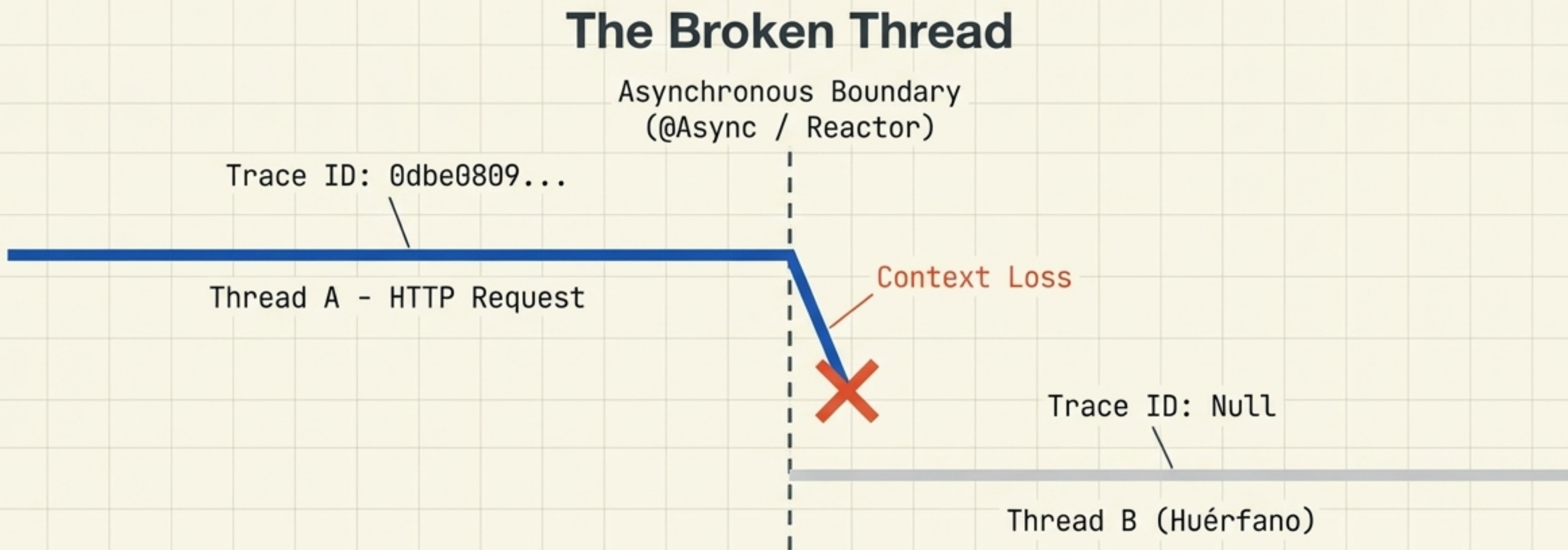
OpenTelemetryAppender

2024-12-23 INFO [0dbe0809731e35081d6db16c2ca0ef91-8aa7bd0083a47082] dev.danvega.ot.Greeting user: World

El Trace ID (0dbe...) vincula este log exacto con la cascada visual en Grafana.

El Peligro Oculto: La Pérdida de Contexto

El contexto de OTel (Trace ID) se almacena en variables ThreadLocal. Cuando Spring cambia de hilo, el hilo nuevo nace ciego. El trace se rompe, rompe, creando Spans huérfanos.



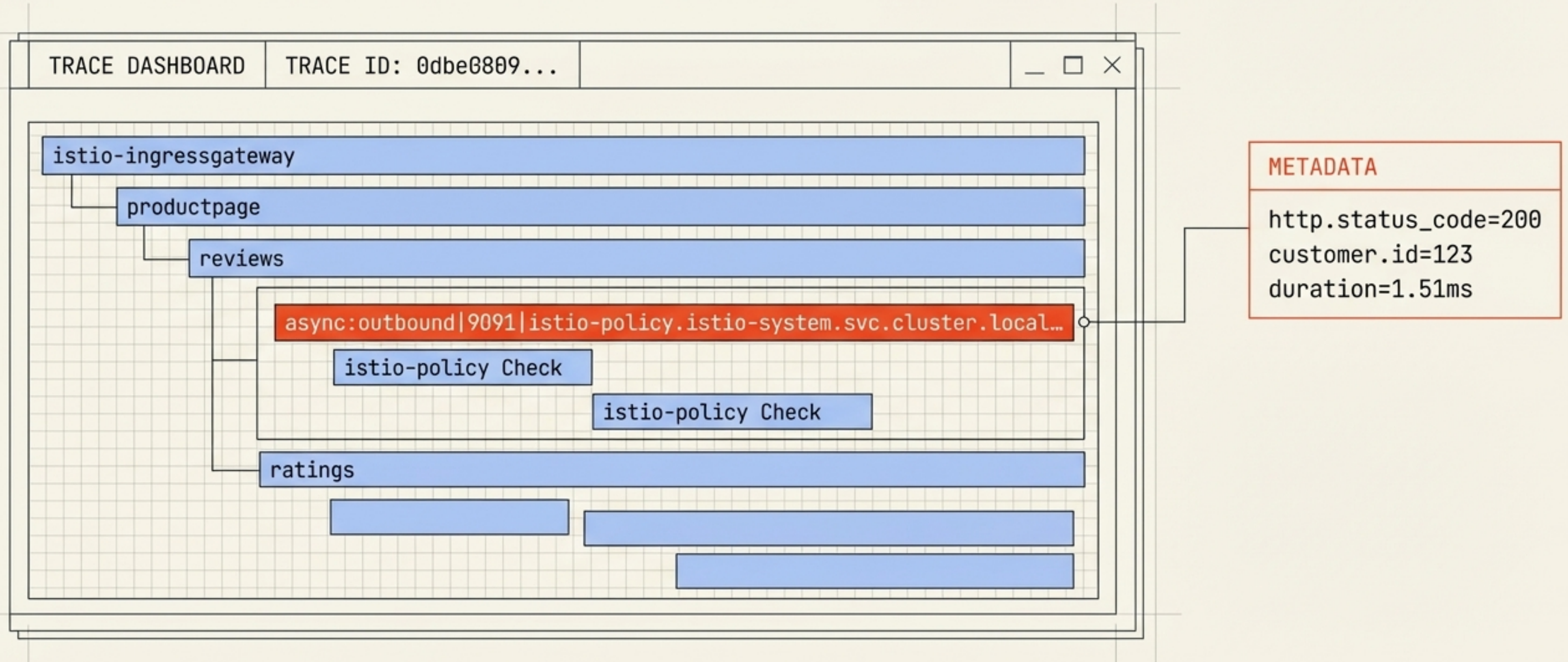
Matriz de Diagnóstico: Reparando el Hilo

Regla de oro: Nunca instanciar clientes manualmente. Usa los constructores inyectados por Spring.

Error	Solución
Escenario 1: Clientes HTTP	
<code>new RestTemplate()</code> o <code>new RestClient()</code> (Pierde los interceptores W3C).	Inyectar <code>RestClient.Builder</code> .
Escenario 2: Spring WebFlux (Reactor)	
Operaciones reactivas por defecto pierden el <code>ThreadLocal</code> .	Añadir <code>Hooks.enableAutomaticContextPropagation()</code> en el método <code>main</code> .
Escenario 3: Métodos Asíncronos (@Async)	
<code>AsyncTaskExecutor</code> crea hilos limpios sin contexto.	Configurar un <code>@Bean</code> de <code>ContextPropagatingTaskDecorator</code> .

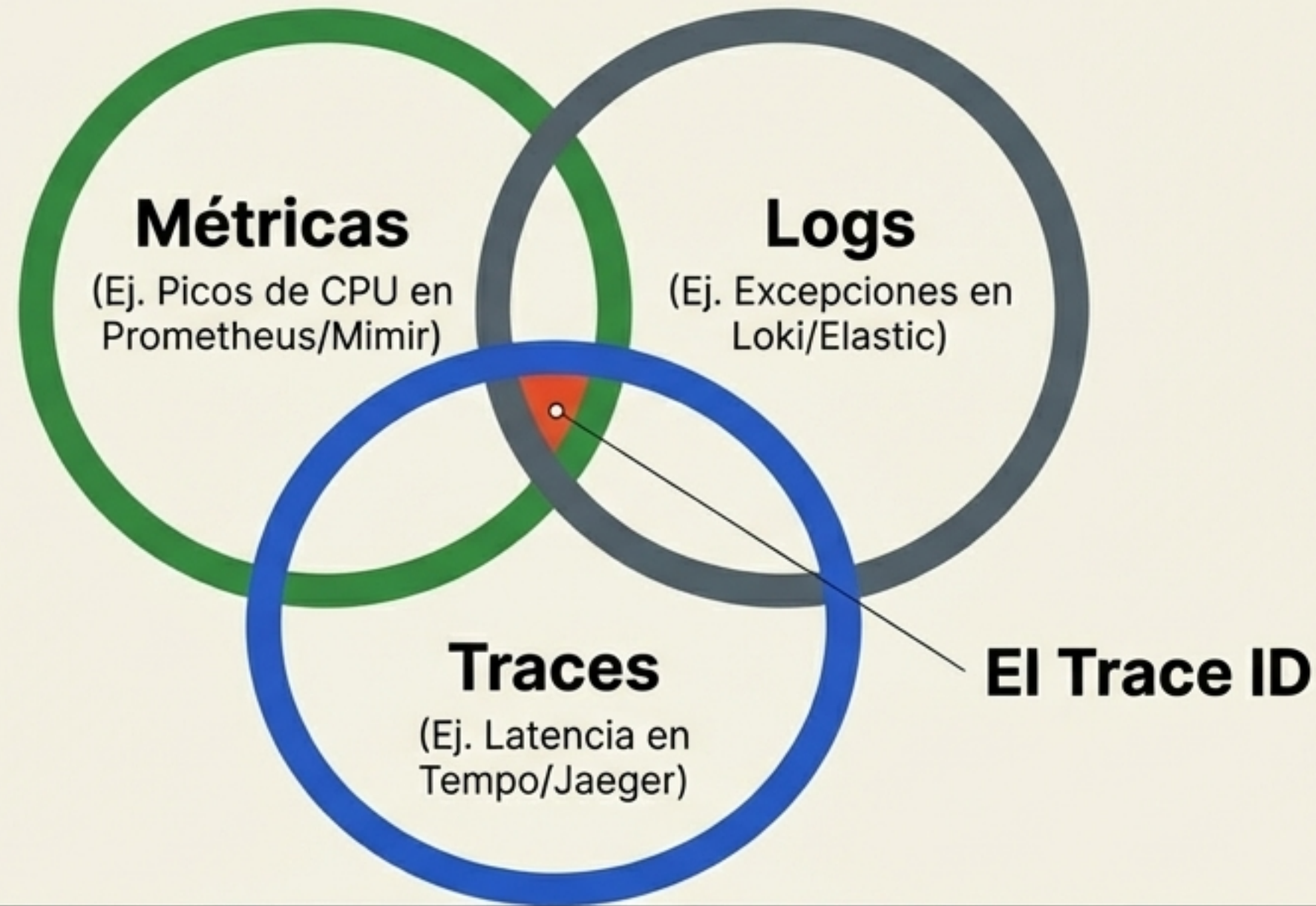
La Recompensa: Trazabilidad Total

La resolución de cuellos de botella pasa de ser una investigación forense de días a una simple inspección visual de segundos.



La Trinidad de la Observabilidad

El verdadero poder de OTLP en Spring Boot 4 es la correlación instantánea.
Un solo identificador une las tres señales.



Desde un pico en una gráfica de métricas, haz clic para ver la traza exacta de ese segundo, y desde esa traza, salta directamente a los logs de error asociados. Cero adivinanzas.

El Plano para Producción

Implementar OpenTelemetry requiere calibración. Lista de verificación antes de ir a producción:

- ☐ **1.** **Ajustar el Muestreo (Sampling):** Cambiar `management.tracing.sampling.probability` de 1.0 a un ratio sostenible (ej. 0.05 o 0.1) para evitar saturar la red y el almacenamiento.
- ☐ **2.** **Limpiar Dependencias Legadas:** Eliminar cualquier rastro de `spring-cloud-sleuth`, puentes de Brave antiguos o el agente JAR de Java.
- ☐ **3.** **Auditar la Creación de Hilos:** Revisar todo el código buscando instanciaciones manuales de clientes HTTP o creación de hilos puros (`new Thread()`) que rompan el contexto.
- ☐ **4.** **Habilitar Soporte Docker Compose:** En desarrollo, usar `spring-boot-docker-compose` con la imagen `grafana/otel-lgtm` para levantar el stack completo automáticamente.